



SIBDAS

SESSÕES e Formulários

**ISEP Ginásio**

Utilizador

Password

Entrar ➔

O username tem que ser um email válido.
O username deve ter entre 5 e 50 caracteres.
A password deve ter entre 6 e 12 caracteres.

Sessões

- 1: **Formulário de Autenticação**
- 2: **Envio de dados Formulário (POST)**
- 3: **Sessões**

1 Formulário de Autenticação

Objetivo: Modularizar a página de login da aplicação ISEP Ginásio, separando o cabeçalho, rodapé e mensagens de erro em ficheiros reutilizáveis, de forma a tornar o código mais organizado, limpo e fácil de manter.

1. Navegar até à pasta onde está o projeto ISEP Ginásio.

2. Abrir o ficheiro login.php, que está localizado em: /isep-ginasio/public/login.php. Abrir esta pasta no **Visual Studio Code** (clicar com o botão direito na pasta e escolher “Abrir com Code”).

3. No início do ficheiro login.php, antes de qualquer conteúdo HTML, escrever:

```
<?php include '../private/includes/header.php'; ?>
```

4. Remover toda a secção original <head> ... </head> e a tag <body> do ficheiro login.php.

5. No final do ficheiro login.php, **após todo o conteúdo HTML**, escrever:

```
<?php include '../private/includes/footer.php'; ?>
```

6. Remover as linhas <script ...></script>, </body> e </html> que estavam originalmente em login.php.



7. No ficheiro login.php localizar esta secção

```

<h2><strong> ISEP Ginásio</strong></h2>
```

Substituir pelo seguinte código:

```

<h2><strong> <?php echo APP_NAME; ?></strong></h2>
```

2 Envio de dados Formulário (POST)

Objetivo: Compreender como os dados introduzidos pelo utilizador num formulário HTML são enviados para o servidor, e como podem ser tratados no PHP através do método POST. Preparar o formulário de login para **enviar os dados inseridos pelo utilizador** para um ficheiro PHP no servidor, usando o método POST.

Etapas do Processo

public/login.php

POST

text_username = ...
text_password =

/private/index.php

login.php

```
<form action="../private/index.php" method="post">
  <label>Username</label>
  <input type="text" name="text_username">
  <label>Password</label>
  <input type="password" name="text_password">
  <input type="submit" value="Entrar">
</form>
```

Especifica a URL do script que receberá e processará os dados inseridos pelo utilizador quando o formulário for enviado

Especifica o método de envio dos dados do formulário. No caso do método POST, os dados são enviados no corpo da requisição HTTP.

Os dados inseridos nos campos do formulário são enviados através das variáveis text_username e text_password

Para aceder aos dados enviados pelo formulário, pode-se usar \$_POST['text_username'] para obter o valor inserido no campo com o atributo name="text_username", e \$_POST['text_password'] para obter o valor do campo com o atributo name="text_password"

1 Abrir o ficheiro login.php (/isep-ginasio/public/login.php) no VSCode

2. Procura a tag <form>. Substituir o caminho no <form> por:

```
<form action="../private/index.php" method="post">
```

3. Atualizar os campos com os atributos **name**. Modifica os campos input para incluírem nomes identificadores:

```
<input type="email" class="form-control" name="text_username" id="">
<input type="password" class="form-control" name="text_password" id="">
```

Objetivo: Modificar o ficheiro PHP que recebe os dados do formulário para verificar se os valores inseridos nos campos **"Utilizador"** e **"Password"** estão a ser corretamente enviados e recebidos via POST.

1. Abrir o ficheiro de destino **/isep-ginasio/private/index.php** no no Visual Studio Code. Este ficheiro irá receber os dados enviados do formulário (login.php)

2. **Inserir o bloco para testar os dados recebidos.** Logo antes dos includes (header.php e nav.php), adicionar o seguinte bloco:

```
<?php
// Mostrar os dados recebidos pelo formulário através do método POST
// O nome dos campos (entre aspas) deve ser igual ao atributo "name" no Login.php

echo "Utilizador: " . $_POST['text_username'] . "<br>"; // Mostra o username inserido
echo "Password: " . $_POST['text_password'];           // Mostra a password inserida

// Em produção, **nunca** se deve mostrar a password assim – isto é apenas para testes!
?>
```

O ficheiro deverá agora ficar assim:

```
<?php
// Mostrar os dados recebidos pelo formulário através do método POST
// O nome dos campos (entre aspas) deve ser igual ao atributo "name" no Login.php

echo "Utilizador: " . $_POST['text_username'] . "<br>"; // Mostra o username inserido
echo "Password: " . $_POST['text_password'];           // Mostra a password inserida

// Em produção, **nunca** se deve mostrar a password assim – isto é apenas para testes!
?>
<?php include 'includes/header.php'; ?>
<?php include 'includes/nav.php'; ?>

<div class="container-fluid">
  <div class="row">

    <?php include 'includes/sidebar.php'; ?>
```

3. Testar o funcionamento. Acede a: <http://localhost/isep-ginasio/public/login.php>

- Preencher o formulário com qualquer **utilizador** e **password**.
- Clicar em **"Entrar"**.
- Será redirecionado para index.php, e deverá ver:

Utilizador: pedro@gmail.com
Password: dsasdasdasd



- `$_POST['text_password']`:
Acede ao valor inserido no campo do formulário com o atributo `name="text_password"`.
- O operador `.` em PHP é usado para **concatenar** strings.
Exemplo: "Utilizador: " . "Ana" → resulta em Utilizador: Ana.
- `<br>`:
É uma **quebra de linha HTML**, usada para apresentar os dois valores em linhas separadas no browser.

Objetivo: Evitar que o ficheiro `index.php` seja acedido diretamente via URL e **permitir execução apenas quando os dados forem enviados por um formulário** (via método POST).

1. Abrir no VSCode o ficheiro `index.php` (`/isep-ginasio/private/index.php`)

2. **Adicionar verificação do tipo de pedido HTTP.** Logo **antes de aceder a `$_POST`**, colocar o seguinte código:

```
<?php
// -----
// SEGURANÇA: Impede que o utilizador aceda diretamente a este script.
// Este ficheiro deve ser acedido apenas através de submissão de formulário (POST).
// Se for acedido diretamente (por URL) recebe a informação de Acesso Inválido
// -----
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    // Se não for uma submissão do formulário, termina o script
    exit('Acesso inválido');
}
?>
```

- `$_SERVER['REQUEST_METHOD']` contém o tipo de pedido HTTP (ex: GET, POST, etc).
- Se o pedido não for POST, o script termina imediatamente com `exit()`.

O ficheiro deverá agora ficar assim:

```
<?php
// -----
// SEGURANÇA: Impede que o utilizador aceda diretamente a este script.
// Este ficheiro deve ser acedido apenas através de submissão de formulário (POST).
// Se for acedido diretamente (por URL) recebe a informação de Acesso Inválido
// -----
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    // Se não for uma submissão do formulário, termina o script
    exit('Acesso inválido');
}

// Mostrar os dados recebidos pelo formulário através do método POST
// O nome dos campos (entre aspas) deve ser igual ao atributo "name" no Login.php

echo "Utilizador: " . $_POST['text_username'] . "<br>"; // Mostra o username inserido
echo "Password: " . $_POST['text_password'];           // Mostra a password inserida

// Em produção, **nunca** se deve mostrar a password assim – isto é apenas para testes!
?>
```

Situação

Resultado

Acesso direto a index.php pela barra de endereço	Mostra Acesso inválido
Submissão do formulário corretamente preenchido	Mostra o username e a password

3. Abrir o browser e inserir a URL diretamente Na barra de endereços do browser, escrever:

<http://127.0.0.1/isep-ginasio/private/index.php>. **Verificar o resultado no ecrã** . Deverá aparecer a seguinte mensagem:

Acesso inválido

Isso confirma que a verificação do método POST está a funcionar corretamente e impede o carregamento do conteúdo se o formulário não for submetido.

4. Agora testar com submissão válida. Abrir o browser e inserir a URL

<http://127.0.0.1/isep-ginasio/public/login.php>

- Preenche o formulário com:
 - **Utilizador:** teste@teste.com
 - **Password:** 123456



ISEP Ginásio

Utilizador

Password

Entrar ➔

- Clica em "Entrar"
- O ficheiro index.php deverá abrir e mostrar:

Utilizador: teste@teste.com

Password: 123456



ISEP Ginásio

Menu

-  Clientes
-  Agendamento de treinos

ISEP Ginásio

Escolhe uma opção no menu lateral para continuar.

Objetivo: Evitar o acesso direto ao ficheiro index.php e, em vez de mostrar uma mensagem, **redirecionar o utilizador automaticamente para a página de login (login.php)**.

1. Abrir no VSCode o ficheiro index.php (/isep-ginasio/private/index.php)
2. Substituir o bloco de verificação existente

Substituir:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    exit('Acesso inválido');
}
```

Por:

```
// -----
// SEGURANÇA: Impede que o utilizador aceda diretamente a este script.
// Este ficheiro deve ser acedido apenas através de submissão de formulário (POST).
// Se for acedido diretamente (por URL), será redirecionado para o login.
// -----
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    // Redireciona para o formulário de login (interface pública)
    header('Location: ../public/login.php');
    // Encerra a execução do script imediatamente após o redirecionamento
    return;
}
```

- Se o pedido não for POST, o utilizador é redirecionado com header().
- return; impede que o restante código continue a ser executado.
- O caminho ../public/login.php funciona porque index.php está dentro da pasta private/.

3. Abrir o browser e inserir a URL diretamente Na barra de endereços do browser, escrever:

<http://127.0.0.1/isep-ginasio/private/index.php>.



Verificar o resultado no ecrã . Deverá aparecer a seguinte mensagem:



The image shows a login form for 'ISEP Ginásio'. At the top, there is a logo of a hand holding a dumbbell and the text 'ISEP Ginásio'. Below this, there are two input fields: 'Utilizador' (Username) and 'Password'. At the bottom, there is a button labeled 'Entrar' (Login) with a right-pointing arrow icon.

Ou seja é redirecionado para a página login.php

Objetivo: Assegurar que os campos `text_username` e `text_password` existem antes de os usar.

1. Abrir no VSCode o ficheiro index.php (/isep-ginasio/private/index.php)

2. Substituir o código de exibição direta

```
echo "Utilizador: " . $_POST['text_username'] . "<br>"; // Mostra o username inserido
echo "Password: " . $_POST['text_password'];           // Mostra a password inserida
```

por

```
// -----
// RECOLHA DE DADOS DO FORMULÁRIO
// -----
// Verifica se o campo 'text_username' foi enviado via POST.
// Se sim, guarda-o na variável $username. Caso contrário, usa string vazia.
$username = isset($_POST['text_username']) ? $_POST['text_username'] : '';
// O mesmo para o campo da password.
$password = isset($_POST['text_password']) ? $_POST['text_password'] : '';

// -----
// APRESENTAÇÃO DE DADOS ENVIADOS
// -----
echo "Utilizador: " . $username . "<br>";
echo "Password: " . $password;
```


- `isset()` garante que o campo existe antes de tentar aceder ao valor.
- Se não existir (ex: alguém removeu o campo), será atribuído um valor vazio ("")

Teste com DevTools (F12) no Google Chrome

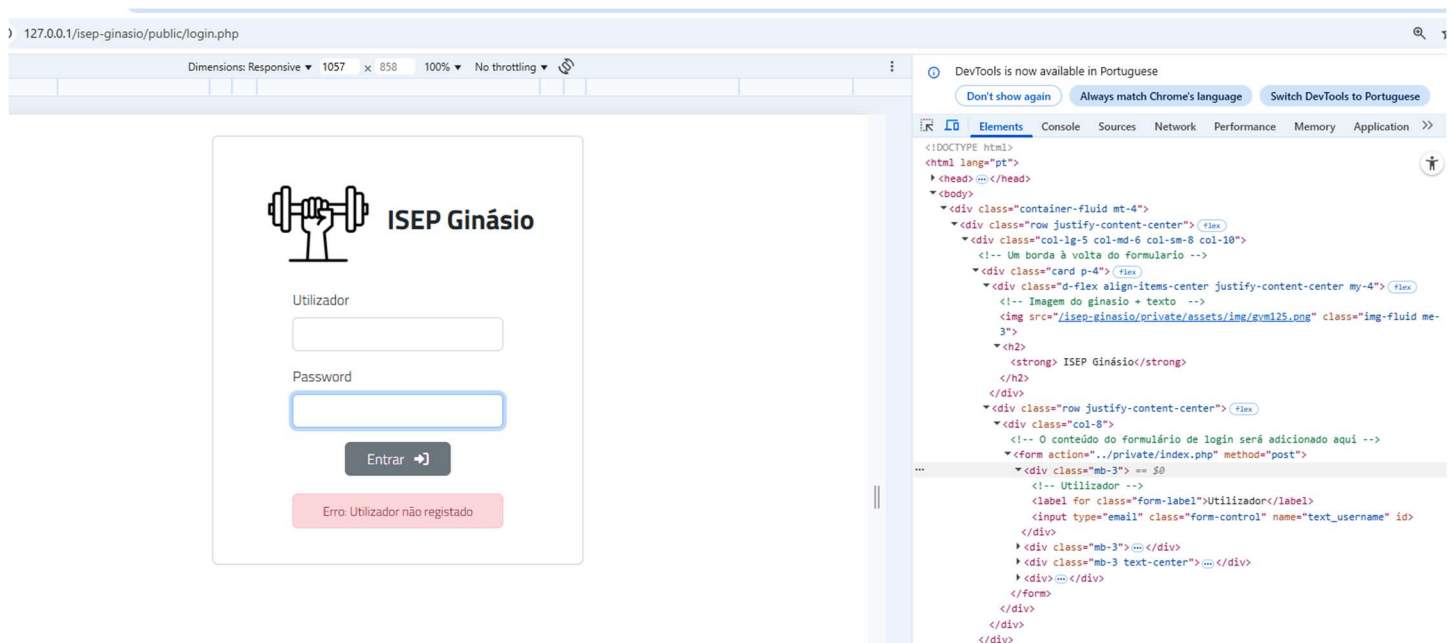
1. Enviar dados normalmente

- Acede a `http://localhost/isep-ginasio/public/login.php`
- Preenche o formulário e clica em "Entrar".
- Verifica que os dados aparecem corretamente no `index.php`.

2 Manipular os dados com F12

- Na página do formulário, carregar em F12 (DevTools).
- Ir ao separador **Elements**.
- Localizar o campo do utilizador:

```
<input type="email" class="form-control" name="text_username" id="">
```



- Alterar o atributo `name="text_username"` para algo diferente, como `name="utilizador_inventado"`.

```
</div>
<div class="row justify-content-center"> flex
  <div class="col-8">
    <!-- O conteúdo do formulário de login será adicionado aqui -->
    <form action="../../private/index.php" method="post">
      <div class="mb-3">
        <!-- Utilizador -->
        <label for class="form-label">Utilizador</label>
        <input type="email" class="form-control" name="utilizador_inventado" id="" > == $0
      </div>
      <div class="mb-3"> ... </div>
      <div class="mb-3 text-center"> ... </div>
      <div> ... </div>
    </form>
```

- Preenche o formulário e clica em "Entrar".

Utilizador

Password

Entrar ➞

Utilizador:

Password: 123456



ISEP Ginásio

Menu

Clientes

Agendamento de treinos

Planos de Treino

Equipamentos

Produtos e Serviços

ISEP Ginásio

Escolhe uma opção no menu lateral para continuar.

3 Sessões

Objetivo: Permitir que mensagens de erro (como dados inválidos ou falhas no login) sejam enviadas para a página de login, utilizando variáveis de sessão (\$_SESSION), e apresentadas ao utilizador após o redirecionamento.

1. Abrir no VSCode o ficheiro **login.php** (/isep-ginasio/public/login.php)
2. Inserir o seguinte bloco PHP no topo do ficheiro, **antes de incluir o header.php**

```
<?php
// Inicia a sessão (necessário para usar $_SESSION)
session_start();

// Inicializa a variável que irá conter os erros de validação
$validation_errors = [];

// -----
// RECOLHA DE MENSAGENS TEMPORÁRIAS DA SESSÃO
// -----

// Verifica se existem erros de validação guardados na sessão
if (!empty($_SESSION['validation_errors'])) {
    // Se existirem, copia-os para a variável local
    $validation_errors = $_SESSION['validation_errors'];

    // Remove os erros da sessão para que não apareçam novamente numa recarga de página
    unset($_SESSION['validation_errors']);
}

// Inicializa a variável que irá conter erros de servidor
$server_error = [];

// Verifica se existe algum erro de servidor guardado na sessão
if (!empty($_SESSION['server_error'])) {
    // Se existir, copia-o para a variável local
    $server_error = $_SESSION['server_error'];

    // Remove o erro da sessão após ser lido
    unset($_SESSION['server_error']);
}
?>
```

Este bloco de código é tipicamente utilizado numa página de login ou formulário para recuperar mensagens de erro temporárias armazenadas na **sessão** (\$_SESSION). Isso permite que o utilizador veja mensagens de erro mesmo depois de ser redirecionado, por exemplo, de volta ao formulário após uma tentativa falhada.

- **validation_errors:** Erros causados por preenchimento incorreto do formulário (ex: campos obrigatórios por preencher, formatos inválidos).
- **server_error:** Erros mais gerais, como falha na ligação à base de dados ou exceções de sistema.

- `session_start();`
 - Inicia a sessão PHP, permitindo aceder às variáveis armazenadas em `$_SESSION`.
- `$validation_errors = [];`
 - Inicializa um array que irá conter erros de validação (por exemplo, campos em branco, credenciais erradas).
- `if (!empty($_SESSION['validation_errors'])) { ... }`

Verifica se existe um array de erros de validação na sessão:

 - Se existir, copia-o para a variável local `$validation_errors`.
 - Depois, remove-o da sessão com `unset()` para não se repetir em acessos seguintes.
- `$server_error = [];`
 - Inicializa a variável que poderá conter um erro técnico, como falha na ligação à base de dados
- `if (!empty($_SESSION['server_error'])) { ... }`

Verifica se existe um erro de servidor armazenado:

 - Se existir, guarda-o na variável `$server_error` e remove-o da sessão.

Boas praticas:

- Uso de `isset()` ou `!empty()` para garantir que a variável existe e contém valor.
- Utilização de `unset()` para evitar que os erros sejam reapresentados em visitas seguintes.
- Separação entre **erros de validação** (relacionados com o utilizador) e **erros de servidor** (problemas técnicos).

Objetivo: Integrar mensagens de erro dinâmicas no formulário de login

1. Substituir bloco de erro fixo do HTML.

Erro: Utilizador não registado

Substituir esta parte:

```
<div>
  <!-- Erros -->
  <div class="alert alert-danger p-2 text-center">
    Erro: Utilizador não registado
  </div>
</div>
```

Por

```
<!-- ----- -->
<!-- APRESENTAÇÃO DE MENSAGENS DE ERRO (VALIDAÇÃO E SERVIDOR) -->
<!-- ----- -->

<!-- Verifica se existem erros de validação -->
<?php if (!empty($validation_errors)) : ?>
  <!-- Se existirem, apresenta um alerta de erro (vermelho) usando as classes do Bootstrap -->
  <div class="alert alert-danger p-2 text-center">

    <!-- Percorre todos os erros de validação -->
    <?php foreach ($validation_errors as $error) : ?>
      <!-- Mostra cada erro dentro de uma <div>, escapando caracteres especiais para segurança -->
      <div><?= htmlspecialchars($error) ?></div>
    <?php endforeach; ?>

  </div>
<?php endif; ?>

<!-- Verifica se existe um erro de servidor -->
<?php if (!empty($server_error)) : ?>
  <!-- Apresenta também num alerta de erro (vermelho) -->
  <div class="alert alert-danger p-2 text-center">
    <!-- Mostra o erro do servidor, também escapado com htmlspecialchars -->
    <div><?= htmlspecialchars($server_error) ?></div>
  </div>
<?php endif; ?>
```

O resultado final do login.php ficará assim

```
<!-- O conteúdo do formulário de login será adicionado aqui -->
<form action="../../../private/index.php" method="post">
  <div class="mb-3">
    <!-- Utilizador -->
    <label for="" class="form-label">Utilizador</label>
    <input type="email" class="form-control" name="text_username" id="">
  </div>

  <div class="mb-3">
    <!-- Password -->
    <label for="" class="form-label">Password</label>
    <input type="password" class="form-control" name="text_password" id="">
  </div>

  <div class="mb-3 text-center">
    <!-- Submit -->
    <button type="submit" class="btn btn-secondary px-4"> Entrar
      <i class="fa-solid fa-right-to-bracket ms-2"></i>
    </button>
  </div>

  <!-- ----- -->
  <!-- APRESENTAÇÃO DE MENSAGENS DE ERRO (VALIDAÇÃO E SERVIDOR) -->
  <!-- ----- -->

  <!-- Verifica se existem erros de validação -->
  <!-- Verifica se existem erros de validação -->
  <?php if (!empty($validation_errors)) : ?>
    <!-- Se existirem, apresenta um alerta de erro (vermelho) usando as classes do Bootstrap -->
    <div class="alert alert-danger p-2 text-center">

      <!-- Percorre todos os erros de validação -->
      <?php foreach ($validation_errors as $error) : ?>
        <!-- Mostra cada erro dentro de uma <div>, escapando caracteres especiais para segurança -->
        <div><?=> htmlspecialchars($error) ?></div>
      <?php endforeach; ?>

    </div>
  <?php endif; ?>

  <!-- Verifica se existe um erro de servidor -->
  <?php if (!empty($server_error)) : ?>
    <!-- Apresenta também num alerta de erro (vermelho) -->
    <div class="alert alert-danger p-2 text-center">
      <!-- Mostra o erro do servidor, também escapado com htmlspecialchars -->
      <div><?=> htmlspecialchars($server_error) ?></div>
    </div>
  <?php endif; ?>

</form>
```

- `if (!empty($validation_errors))`
Verifica se existem mensagens de erro de validação armazenadas na variável `$validation_errors`. Esta variável é normalmente preenchida a partir da `$_SESSION['validation_errors']` no topo do ficheiro.
- `foreach ($validation_errors as $error)`
Itera sobre o array de erros e apresenta cada erro dentro de uma `div`.
- `<?=> htmlspecialchars($error) ?>`
Utiliza a função `htmlspecialchars()` para evitar a execução de código malicioso (XSS), convertendo caracteres especiais em entidades HTML.

- `if (!empty($server_error))`
Verifica se existe uma mensagem de erro de servidor (por exemplo, falha de ligação à base de dados), e exibe-a de forma semelhante

Objetivo: Verificar se os dados inseridos no formulário (username e password) são válidos. Se não forem, redirecionar o utilizador para a página de login e apresentar os erros com mensagens informativas.

1. Abrir no VSCode o ficheiro **index.php** (/isep-ginasio/private/index.php)
2. Iniciar sessão. Colocar a instrução logo no início.

```
// Inicia a sessão para poder usar a variável $_SESSION
session_start();
```

Antes de qualquer uso de `$_SESSION`, é obrigatório iniciar a sessão com `session_start()`. Isto permite guardar informações temporárias entre páginas, como erros de validação

3. Verificação do Tipo de Acesso (Segurança). Código já implementado

```
if ($_SERVER['REQUEST_METHOD'] != 'POST') {
    header('Location: ../public/login.php');
    return;
}
```

Garante que este script só é **acedido após envio de formulário**. Se alguém tentar aceder diretamente via URL, é **redirecionado** para o login. O return evita que o script continue a correr

```
$username = isset($_POST['text_username']) ? $_POST['text_username'] : '';
$password = isset($_POST['text_password']) ? $_POST['text_password'] : '';
```

Verifica se os campos `text_username` e `text_password` foram submetidos. Se foram, guarda os valores nas variáveis correspondentes. Se não foram, atribui uma string vazia (").

3. Validar os dados (sanitização)

```
// -----  
// VALIDAÇÃO DOS DADOS  
// -----  
  
// Inicializa um array vazio para guardar mensagens de erro de validação  
$validation_errors = [];  
  
// Verifica se o nome de utilizador (username) é um endereço de email válido  
// Se não for, adiciona uma mensagem de erro ao array  
if (!filter_var($username, FILTER_VALIDATE_EMAIL)) {  
    $validation_errors[] = 'O username tem que ser um email válido.';  
}  
  
// Verifica se o nome de utilizador tem um comprimento entre 5 e 50 caracteres  
// Isto evita usernames demasiado curtos ou excessivamente longos  
if (strlen($username) < 5 || strlen($username) > 50) {  
    $validation_errors[] = 'O username deve ter entre 5 e 50 caracteres.';  
}  
  
// Verifica se a password tem um comprimento entre 6 e 12 caracteres  
// Garante uma password minimamente segura, mas fácil de recordar  
if (strlen($password) < 6 || strlen($password) > 12) {  
    $validation_errors[] = 'A password deve ter entre 6 e 12 caracteres.';  
}
```

- Verifica se o username é um email válido com filter_var(...).
- Verifica se o username tem comprimento entre 5 e 50 caracteres.
- Verifica se a password tem comprimento entre 6 e 12 caracteres.
- Se algum critério falhar, o erro correspondente é adicionado ao array \$validation_errors.

4. Se houver erros, guardar na sessão e redirecionar

```
// Se existirem erros de validação, guarda-os na sessão  
// Depois, redireciona o utilizador de volta para o formulário de login  
if (!empty($validation_errors)) {  
    $_SESSION['validation_errors'] = $validation_errors;  
  
    // Redireciona para a página de login (ou outro formulário)  
    header('Location: ../public/login.php'); // ou 'login_form.php'  
  
    // Encerra o script para impedir execução posterior  
    return;  
}
```

- Se foram encontrados erros de validação, são guardados na \$_SESSION.
- Em seguida, o utilizador é redirecionado novamente para o formulário de login.
- O script termina com return para não continuar o processamento.

Input inválido	Resultado
Email mal formatado	Mostra "O username tem que ser um email válido."
Password com 3 caracteres	Mostra "A password deve ter entre 6 e 12 caracteres."
Tudo correto	Continua para o próximo passo (verificação de credenciais)

4. Abrir o formulário de login

- Abrir o navegador (ex: Chrome).
- Aceder à seguinte URL: <http://localhost/isep-ginasio/public/login.php>

5. Submeter campos vazios

- Clicar diretamente em **“Entrar”** sem preencher nenhum campo.
- **Resultado esperado:**

O username tem que ser um email válido.
O username deve ter entre 5 e 50 caracteres.
A password deve ter entre 6 e 12 caracteres.

6. Submeter um email inválido

- No campo **Utilizador**, escrever:
teste123@teste
- No campo **Password**, escrever:
123456
- Clicar em **Entrar**
- **Resultado esperado:**

O username tem que ser um email válido.

7. Submeter uma password curta

- No campo **Utilizador**, escrever:
teste@isep.pt
- No campo **Password**, escrever:
123
- Clicar em **Entrar**
- **Resultado esperado:**

A password deve ter entre 6 e 12 caracteres.

7. Submeter dados corretos (login válido)

- No campo **Utilizador**, escrever:
ana@isep.pt
- No campo **Password**, escrever:
123456
- Clicar em **Entrar**
- **Resultado esperado:**
 - O utilizador entra na área privada (private/index.php)
 - O nome do utilizador pode ser mostrado na barra de navegação

Utilizador: ana@isep.pt
Password: 123456



ISEP Ginásio

Resumo do Fluxo (private/index.php)

1. **Formulário é submetido** → este script é chamado.
2. **Verifica-se o tipo de pedido** → só continua se for POST.
3. **Recolhem-se os dados** → username e password.
4. **Valida-se os dados** → email válido, comprimentos mínimos/máximos.
5. **Se houver erros**, são guardados na \$_SESSION → redireciona para login.
6. **Se estiver tudo certo**, o processo continua (verificação de credenciais, sessão, etc.).

Objetivo: Simular o comportamento de uma base de dados que verifica se o login é válido ou não.

O resultado da “consulta” é armazenado num array associativo \$result, como se tivesse sido retornado de uma base de dados.

1 Criar a variável de resultado simulado

Coloca esta linha depois da validação dos dados, antes de guardar o utilizador na sessão:

```
// -----  
// SIMULAÇÃO DE RESULTADO DE LOGIN (antes da ligação real à base de dados)  
// -----  
  
// Simula o resultado que viria de uma verificação à base de dados  
// Neste caso, assume-se que o Login é válido (status = 1)  
// Mais tarde, esta variável será substituída por um resultado real vindo da BD  
$result['status'] = 1; // 1 = Login válido, 0 = inválido
```

2. Verificar o estado da autenticação

A seguir, faz a verificação:

```
// Verifica se o status retornado indica login inválido
if (!$result['status']) {
    // Se o login for inválido, guarda uma mensagem de erro na sessão
    $_SESSION['server_error'] = 'Login inválido';

    // Redireciona o utilizador novamente para o formulário de login
    header('Location: ../public/login.php'); // ou 'login_form.php'

    // Encerra o script para não continuar o processamento
    return;
}

// Se o status for 1 (válido), o código continuará – aqui será futuramente criada a sessão
do utilizador e o redirecionamento para a área privada.
```

O que este bloco faz:

- Reserva o local onde, futuramente, será feita a validação do utilizador com base em dados reais (ex: MySQL).
- Permite simular o fluxo completo de login **antes de implementar a base de dados**.
- Já integra a gestão de erro via \$_SESSION e redirecionamento, tal como será feito na versão final.

Valor de \$result['status']	Resultado
0	Redireciona com mensagem: “Login inválido”
1	Sessão é criada e utilizador acede a /private/index.php

Nota técnica complementar

Esta abordagem simula o que normalmente será feito com uma consulta SQL como:

```
SELECT * FROM utilizadores WHERE email = ? AND password = ?
```

O resultado dessa consulta (se o utilizador existir e a password corresponder) é representado aqui por \$result['status'] = 1.

3. Simular login inválido

- Editar no ficheiro login_submit.php a linha: `$result['status'] = 0;`
- Submeter dados válidos
- Resultado esperado:

Login inválido

- Editar no ficheiro login_submit.php a linha `$result['status'] = 1;`

Objetivo: Mostrar o email do utilizador (ou outro identificador) no topo da aplicação após o login, utilizando a variável de sessão `$_SESSION['utilizador']`.

1. Adicionar este bloco **logo após** a verificação de que o login é válido:

```
// -----  
// LOGIN BEM-SUCEDIDO: Guardar o utilizador na sessão  
// -----  
  
// Guarda o nome de utilizador na sessão para identificar o utilizador autenticado  
$_SESSION['utilizador'] = $username;  
  
// Agora código da área privada
```

Resumo do Fluxo de Autenticação

1. O formulário de login é submetido

- O utilizador preenche o username (email) e a password e submete o formulário.
- A submissão chama este script através de `method="POST"`.

2. Verifica-se o tipo de pedido HTTP

- O script verifica se foi feito um pedido POST.
- Se for acedido diretamente por URL (GET), redireciona imediatamente para o login.php.

3. Recolhem-se os dados do formulário

- Os valores dos campos `text_username` e `text_password` são recolhidos da variável `$_POST`.
- Se algum deles não for enviado, é atribuído um valor por defeito (string vazia).

4. Valida-se a informação recebida

- O username é validado para garantir que é um endereço de email.
- O comprimento do username e da password é verificado.
- Caso haja erros, estes são guardados em `$_SESSION['validation_errors']`.

5. Se existirem erros de validação

- O utilizador é redirecionado novamente para o formulário de login.
- Os erros podem ser exibidos com `$_SESSION['validation_errors']` na página de login.

6. Simula-se o resultado da verificação na base de dados

- Neste ponto, simula-se uma resposta como se tivesse vindo da base de dados (login válido ou inválido).

- Se o login for inválido (status = 0), guarda-se a mensagem Login inválido em `$_SESSION['server_error']` e redireciona-se para o formulário.

7. Se o login for válido

- Cria-se a variável de sessão `$_SESSION['utilizador']` com o valor do username.
- Esta variável identifica o utilizador autenticado e pode ser usada em qualquer página da área privada.

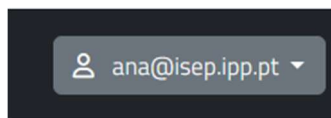
8. Apresentação (temporária) dos dados enviados

- Apenas para testes, são exibidos o username e a password no ecrã.
- Isto será removido mais tarde em ambiente de produção.

9. Inclui-se a navegação e cabeçalho da área privada

- Inclui-se `header.php` e `nav.php`, indicando que o utilizador já está autenticado e pode aceder ao conteúdo privado.

Objetivo: Colocar o nome do utilizador já validado no navbar



1. No início do ficheiro `nav.php`, iniciar a sessão

```
<?php
// Verifica se a sessão ainda não foi iniciada
if (session_status() == PHP_SESSION_NONE) {
    session_start(); // Inicia a sessão
}
// Verifica se o utilizador está autenticado
if (!isset($_SESSION['utilizador'])) {
    // Se não estiver autenticado, redireciona para o formulário de Login
    header('Location: ../public/login.php');
    exit; // Encerra o script
}
// A partir daqui, o utilizador está autenticado
// Podemos usar livremente os dados da sessão
$nome = $_SESSION['utilizador'];
?>
```

2. Substituir o texto "Utilizador" pelo valor da variável `$user`

No botão de utilizador, altera:

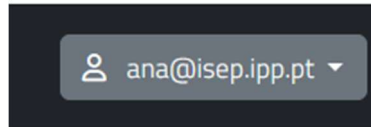
```
<i class="fa-regular fa-user me-2"></i> Utilizador
```

por

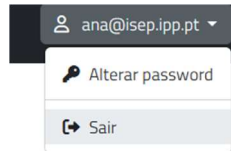
```
<i class="fa-regular fa-user me-2"></i> <?= htmlspecialchars($nome) ?>
```

3. Verificar se a sessão foi criada

- Ir a `private/index.php`
- Verificar se está a aparecer:
Utilizador: `ana@isep.pt`



Objetivo: Permitir que o utilizador termine a sua sessão de forma segura, **removendo todas as variáveis de sessão** e redirecionando-o para o formulário de login.



1. Criar o ficheiro `logout.php`. Cria um novo ficheiro em: `/isep-ginasio/public/logout.php`

2. Adicionar o seguinte conteúdo:

```
<?php
// Inicia a sessão para aceder e manipular os dados da $_SESSION
session_start();

// -----
// TERMINAR A SESSÃO
// -----

// Remove todas as variáveis da sessão
// Isto limpa os dados armazenados, como $_SESSION['utilizador'], etc.
session_unset();

// Destroi completamente a sessão no servidor
// Isto elimina o identificador da sessão e os dados associados
session_destroy();

// -----
// REDIRECIONAMENTO PARA O LOGIN
// -----

// Após terminar a sessão, redireciona o utilizador para a página de login
header('Location: login.php');

// Encerra a execução do script
return;
```

- `session_start();`
Inicia a sessão atual para poder modificá-la.

- `session_unset();`
Remove todas as variáveis de sessão.
- `session_destroy();`
Termina a sessão completamente no servidor.
- `header('Location: ...');`
Redireciona o utilizador para a página de login.

3. Atualizar o link "Sair" no nav.php

Localiza o item do menu dropdown:

```
<li><a class="dropdown-item" href="/isep-ginasio/public/logout.php"><i
```

4. Testar Logout

- Clicar no botão “Sair” no topo da página privada.
- Verifica se és redirecionado para a página de login.
- Tenta aceder diretamente a: `http://localhost/isep-ginasio/private/index.php`
- **Resultado esperado:**
 - É redirecionado para o formulário de login automaticamente.

Objetivo: Cria um ficheiro chamado `funcoes.php` dentro da pasta `includes/`. Este ficheiro irá conter funções reutilizáveis relacionadas com a gestão de sessões. O objetivo deste ficheiro é centralizar funções relacionadas com a gestão de sessões, permitindo iniciar a sessão apenas quando necessário, verificar se o utilizador está autenticado, redirecionar automaticamente para a página de login se não estiver autenticado, e reutilizar estas funções em qualquer página privada com um simples `require_once`.

1. **Criar o ficheiro** No diretório `private/includes/`, cria um ficheiro chamado `funcoes.php`

2. Adicionar o seguinte código ao ficheiro `funcoes.php`

```
// Inicia a sessão se ainda não estiver iniciada
function start_session()
{
    if (session_status() == PHP_SESSION_NONE) {
        session_start();
    }
}
// Verifica se a sessão do utilizador está ativa
function check_session()
{
    return isset($_SESSION['utilizador']);
}
```

```
// Redireciona automaticamente se não houver sessão iniciada
function redirect_if_not_logged($redirect_to = '../public/login.php')
{
    start_session();
    if (!check_session()) {
        header("Location: $redirect_to");
        exit;
    }
}

function logout_and_redirect($redirect_to = '../public/login.php')
{
    start_session();
    session_unset();
    session_destroy();
    header("Location: $redirect_to");
    exit;
}
```

3. Confirmar que o ficheiro está a funcionar Na página `private/index.php` no início do ficheiro, substituir isto

```
session_start();
```

por

```
require_once 'includes/funcoes.php';
start_session();
```

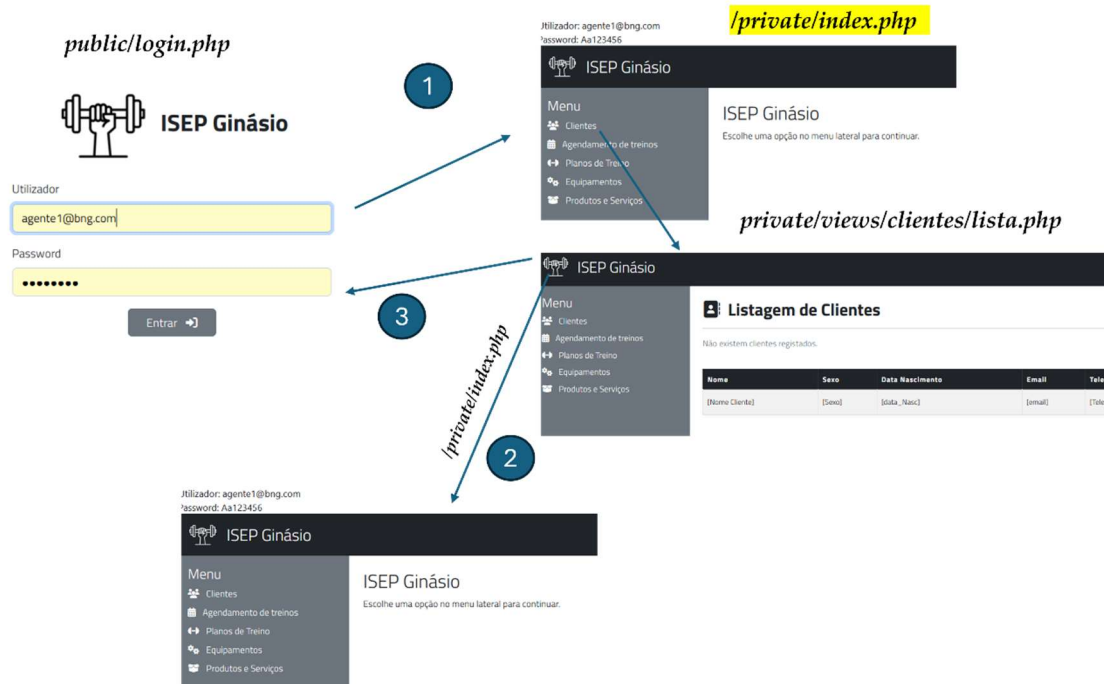
4. Testar o funcionamento

Objetivo: Reestruturar o código para que:

- Um ficheiro (`processa_login.php`) seja responsável **apenas por validar os dados do formulário e iniciar a sessão**;
- Outro ficheiro (`home.php`) seja responsável por **apresentar o conteúdo da área privada**, caso o utilizador já esteja autenticado.

O ficheiro `private/index.php` pode ser acedido de duas formas:

1. **Via login** — o utilizador preenche o formulário, e os dados são recebidos por POST.
2. **Diretamente via URL ou ao circular entre páginas** — por exemplo, se o utilizador aceder diretamente a `private/index.php`.



Qual é o problema?

Se o utilizador já tiver sessão iniciada e tentar aceder a este ficheiro diretamente (sem POST), o código **faz um redirecionamento para o login**, mesmo que ele esteja autenticado. Isto acontece por causa deste bloco:

```
if ($_SERVER['REQUEST_METHOD'] != 'POST') {
    header('Location: ../public/login.php');
    return;
}
```

Como resolver este problema?

- Manter a validação de login via POST com segurança;
- E permitindo que, após o login, o utilizador possa navegar normalmente entre páginas privadas (ex: home.php, clientes.php, equipamentos.php) **sem passar novamente por este ficheiro**.

Estrutura de ficheiros sugerida

```
Isep-ginasio/
├── public/
│   └── login.php
├── private/
│   ├── processa_login.php
│   └── home.php
```

/private/index.php

```
<?php
require_once 'includes/funcões.php';
start_session();
. . .

$_SESSION['utilizador'] = $username;

include 'includes/header.php';
include 'includes/nav.php';
. . .

<?php include 'includes/footer.php'; ?>
```

/private/processa_login.php

```
<?php
require_once 'includes/funcões.php';
start_session();
. . .

$_SESSION['utilizador'] = $username;

// Redirecionar para a página principal privada
header('Location: home.php');
exit;
```

/private/home.php

```
include 'includes/header.php';
include 'includes/nav.php';
. . .

<?php include 'includes/footer.php'; ?>
```

1. Criar o ficheiro processa_login.php

Este ficheiro irá receber os dados do formulário, validar o login e criar a sessão.

```
<?php

require_once 'includes/funcões.php';

start_session();

// -----
// SEGURANÇA: Impede que o utilizador aceda diretamente a este script.
// Este ficheiro deve ser acedido apenas através de submissão de formulário (POST).
// Se for acedido diretamente (por URL), será redirecionado para o Login.
// -----
if ($_SERVER['REQUEST_METHOD'] != 'POST') {
    // Redireciona para o formulário de Login (interface pública)
    header('Location: ../public/login.php');
    // Encerra a execução do script imediatamente após o redirecionamento
    return;
}

// -----
// RECOLHA DE DADOS DO FORMULÁRIO
// -----

. . .

// Guarda o nome de utilizador na sessão para identificar o utilizador autenticado
$_SESSION['utilizador'] = $username;

// Redirecionar para a página principal privada
header('Location: home.php');
exit;
```

2. Criar o ficheiro `home.php` (página inicial privada). Este ficheiro só será acessível por utilizadores autenticados.

```
<?php
require_once 'includes/funcoes.php';
redirect_if_not_logged();
start_session();

$success_message = $_SESSION['success_message'] ?? '';
unset($_SESSION['success_message']);

?>

<?php
include 'includes/header.php';
include 'includes/nav.php';

?>

<?php if (!empty($success_message)) : ?>
<div class="position-fixed top-0 end-0 p-3" style="z-index: 11">
  <div id="toastSuccess" class="toast align-items-center text-bg-success border-0 show" role="alert">
    <div class="d-flex">
      <div class="toast-body">
        <?= htmlspecialchars($success_message) ?>
      </div>
      <button type="button" class="btn-close btn-close-white me-2 m-auto" data-bs-dismiss="toast"></button>
    </div>
  </div>
</div>
<?php endif; ?>

<div class="container-fluid">
  <div class="row">

    <?php include 'includes/sidebar.php'; ?>

    <!-- Conteúdo Principal -->

    <main class="col-md-9 col-lg-10 p-4">
      <section>
        <h2>ISEP Ginásio</h2>
        <p>Escolhe uma opção no menu lateral para continuar.</p>
      </section>
    </main>

  </div>
</div>

<?php include 'includes/footer.php'; ?>
```

3. Atualizar `public/login.php`

```
<form action="../../private/processa_login.php" method="post">
```

4. Atualizar `private/includes/nav.php`

```
<a href="/isep-ginasio/private/index.php">
```

Para

```
<a href="/isep-ginasio/private/home.php">
```